



22. Python: Libraries

"Why build everything yourself when Python already has the tools ready?"

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 21. Python: Exception Handling](#)

Contents:

[1. Built-In Libraries](#)

[Common Built-In Libraries for Beginners](#)

[2. External Libraries](#)

[Common External Libraries](#)

[Summary](#)

[References](#)

One thing that makes Python different from many other programming languages is its massive ecosystem of libraries. Instead of writing every feature from scratch, we can simply import tools that already exist and focus on solving problems faster.

Python follows a philosophy called the Zen of Python, written by Tim Peters. One famous line says: ***"Simple is better than complex."***

Libraries help us follow that idea. Rather than creating our own math engine, calendar system, or networking tool, we can reuse trusted modules that are already tested and optimized.

A library is a collection of prewritten code that helps us perform specific tasks. Libraries allow us to:

- Work with files

- Perform mathematical calculations
- Handle dates and time
- Build graphical interfaces
- Connect to the internet
- Process data
- And much more

Python libraries save time, reduce errors, and make programming much more efficient.

1. Built-In Libraries

Built-in libraries, also known as the Standard Library (or simply modules), are included with every Python installation. This means we can use them immediately without downloading anything extra.

To use a library, we import it into our program. This practice is called: **Importing Libraries**

- Import the entire module

```
import math
```

- Import a specific method

```
from math import sqrt
```

- Import with an alias

```
import datetime as dt
```

Common Built-In Libraries for Beginners

Library	Purpose
<code>array</code>	Work with arrays

Library	Purpose
<code>calendar</code>	Display and manage calendars
<code>collections</code>	Advanced container data types
<code>datetime</code>	Handle dates and time
<code>email</code>	Process email messages
<code>encodings</code>	Handle text encoding systems
<code>functools</code>	Functional programming tools
<code>glob</code>	Find files using patterns
<code>graphviz</code>	Create graph visualizations
<code>hashlib</code>	Generate secure hashes
<code>heapq</code>	Heap queue algorithms
<code>importlib</code>	Advanced importing tools
<code>inspect</code>	Inspect live objects and code
<code>ipaddress</code>	Work with IP addresses
<code>json</code>	Read and write JSON data
<code>keyword</code>	Access Python keywords
<code>linecache</code>	Read specific lines from files
<code>logging</code>	Create log messages
<code>marshal</code>	Serialize Python objects
<code>math</code>	Mathematical functions
<code>multiprocessing</code>	Run multiple processes
<code>numbers</code>	Numeric abstract base classes
<code>operator</code>	Functional operators
<code>os</code>	Interact with the operating system
<code>pathlib</code>	Work with file paths
<code>pickle</code>	Serialize Python objects
<code>pickletools</code>	Analyze pickle data
<code>pstats</code>	Analyze profiling statistics
<code>pydoc</code>	Generate Python documentation
<code>queue</code>	Queue data structures
<code>random</code>	Generate random values
<code>re</code>	Regular expressions

Library	Purpose
<code>sqlite3</code>	SQLite database support
<code>statistics</code>	Statistical calculations
<code>string</code>	String constants and helpers
<code>struct</code>	Convert between Python values and C structures
<code>sys</code>	System-specific parameters
<code>stat</code>	File status information
<code>tkinter</code>	GUI application development
<code>tokenize</code>	Python token processing
<code>typing</code>	Type hinting support
<code>turtle</code>	Beginner graphics and drawing
<code>types</code>	Dynamic type utilities
<code>unicodedata</code>	Unicode character database
<code>urllib</code>	URL handling
<code>uuid</code>	Generate unique identifiers
<code>venv</code>	Create virtual environments
<code>warnings</code>	Warning control
<code>wave</code>	Read and write WAV audio files
<code>xml</code>	XML processing
<code>zipfile</code>	ZIP archive handling
<code>zipapp</code>	Create executable ZIP applications
<code>zoneinfo</code>	Time zone support

- Example: Using a Built-In Library → the math library

```
import math

print(math.sqrt(25))
print(math.pi)
```

```
5.0
3.141592653589793
```

Why Built-In Libraries Matter

Built-in libraries are one of Python's biggest strengths because they:

- Reduce development time
- Provide tested and reliable code
- Make programs shorter and cleaner
- Allow beginners to build powerful applications quickly

Instead of reinventing tools, we simply import and use them.

More Built-In Libraries

Python contains many more built-in modules than the ones listed above. More can be explored here:

https://www.w3schools.com/python/python_ref_modules.asp

2. External Libraries

Now we move on to the bigger world of Python libraries: External Libraries.

Unlike built-in libraries, these are not included automatically with Python. Python can still use them perfectly fine, but first, we need to install them ourselves.

Think of it like installing an application on your computer. Before you can use Photoshop, Discord, or VSCode, you first need to install them. External Python libraries work the same way.

To install a library, we usually use:

```
pip install something
```

For example:

```
pip install numpy
```

After installation, we can import and use it inside Python programs.

Library Name vs Import Name

One thing beginners often find confusing is this: The name used during installation is sometimes different from the name used during importing.

For example:

Install Name	Import Name
<code>pillow</code>	<code>PIL</code>
<code>scikit-learn</code>	<code>sklearn</code>
<code>opencv-python</code>	<code>cv2</code>
<code>spatialmath-python</code>	<code>spatialmath</code>

So always remember:

- `pip install ...` installs the package
- `import ...` loads the module into Python

These names are often similar, but not always identical.

Common External Libraries

Here are some of the most popular external libraries used in Python today.

Library	Purpose
<code>NumPy</code>	Numerical computing and arrays
<code>SciPy</code>	Scientific and mathematical operations
<code>SymPy</code>	Symbolic mathematics
<code>PyGame</code>	Game development
<code>Scikit-Learn</code>	Machine Learning
<code>TensorFlow</code>	Deep Learning and AI
<code>Keras</code>	High-level neural network API
<code>PyTorch</code>	Deep Learning framework
<code>Pandas</code>	Data analysis and dataframes
<code>Librosa</code>	Audio and music analysis
<code>PyDub</code>	Audio processing
<code>OpenCV</code>	Computer Vision and image processing

Library	Purpose
RoboticsToolbox	Robotics algorithms and simulations
SpatialMath	Spatial mathematics and transformations
Matplotlib	Data visualization and plotting
Seaborn	Advanced statistical visualization
StatsModels	Statistical modeling
Pillow	Image processing
Django	Full-stack web development
Flask	Lightweight web framework
BeautifulSoup	HTML and web scraping
Scrapy	Advanced web scraping
requests	HTTP requests and APIs
socket	Network communication
SQLAlchemy	Database ORM
sqlite3	SQLite database management
PyQt	GUI desktop applications
NLTK	Natural Language Processing
Plotly	Interactive visualization
FastAPI	Modern API development
Pydantic	Data validation
Ruff	Python linting and formatting
Rich	Beautiful terminal output

- Example: Installing and Using an External Library → Install NumPy

```
pip install numpy
```

- Use NumPy

```
import numpy as np

numbers = np.array([1, 2, 3, 4, 5])

print(numbers)
print(numbers.mean())
```

```
[1 2 3 4 5]
3.0
```

Why External Libraries Matter

External libraries are one of the reasons Python became so popular.

Instead of building everything from scratch, developers around the world create specialized tools that everyone can reuse. This means we can:

- Build AI systems
- Create websites
- Analyze massive datasets
- Process images and audio
- Develop games
- Automate tasks
- And much more

All without reinventing the wheel.

Summary

Python libraries are one of the biggest strengths of the language.

Built-in libraries give us immediate access to essential tools like math, file handling, randomness, regular expressions, databases, and graphical interfaces. They come included with Python and are ready to use instantly.

External libraries expand Python even further. With just a simple installation command, we gain access to powerful technologies for: Artificial Intelligence; Machine Learning; Data Science; Web Development; Automation; Robotics;

Computer Vision; Audio Processing; Scientific Computing; And many other fields.

This is what makes Python so flexible. We are not limited to one domain or one type of project. With libraries, Python becomes a language that can adapt to almost anything.

And ultimately, that is the real power of Python: ***Not just the language itself, but the enormous ecosystem built around it.***

References

<https://docs.python.org/3/library/index.html>

<https://www.geeksforgeeks.org/python/libraries-in-python/>

<https://www.geeksforgeeks.org/python/external-modules-in-python/>

Next Section:

</> [23. Python: Classes and Objects](#)